Despliegue en GKE y escaneo de seguridad de imágenes con Trivy

Prácticas Unit 4 (Kubernetes) y Unit 5 (Security Images Scanning)

Javier Escobar Serrano
Máster en Inteligencia Artificial — Ingeniería de Modelos 2026-2027

5 de noviembre de 2026

Resumen

Este informe documenta dos prácticas encadenadas sobre el repositorio `JES0406/mlops-practica-icai`. En la primera (Unit 4) se despliega la API de predicción (Flask) y la interfaz web (Streamlit) del modelo Iris en un clúster de Google Kubernetes Engine (GKE) desde GitHub Actions, con las imágenes almacenadas en `gcr.io`. En la segunda (Unit 5) se añade un escaneo de vulnerabilidades con Trivy entre la construcción y la subida de cada imagen, de modo que una vulnerabilidad CRITICAL bloquea el `docker push` y, con él, todo el despliegue. Se muestra el pipeline funcionando, bloqueando una dependencia vulnerable (`pyarrow 13.0.0`, CVE-2023-47248) y volviendo a desplegar tras corregirla. Todo el trabajo está en la rama `p6-k8s-trivy`.

Índice

1. Entorno y punto de partida **2**

2. Práctica Unit 4: despliegue en GKE **2**
2.1. Configuración de Google Cloud . . . 2
2.2. Variables y secretos en GitHub . . . 2
2.3. Manifiestos de Kubernetes . . . 3
2.4. Job `deploy-to-gke` . . . 4
2.5. Verificación . . . 7

3. Práctica Unit 5: escaneo de imágenes con Trivy **8**
3.1. Puerta de seguridad en el pipeline . . . 8
3.2. Etapa A: escaneo con las dependencias actuales . . . 8
3.3. Etapa B: dependencias vulnerables (la puerta bloquea) . . . 9
3.4. Etapa C: corrección de `requirements.txt` . . . 9
3.5. Escaneo local (CRITICAL y HIGH) . . . 10

4. Problemas encontrados **10**

5. Resumen de ejecuciones **10**

6. Costes y limpieza **10**

7. Conclusiones **11**

1. Entorno y punto de partida

- **Repositorio:** fork `JES0406/mlops-practica-icai` de `juanviz/mlops-practica-icai`, con el pipeline DVC + CML (Unit 3) y las imágenes Docker de API y web (Unit 4, Docker).
- **Rama de trabajo:** `p6-k8s-trivy`. Cada etapa se subió como un *push* independiente para que tenga su propia ejecución en GitHub Actions (tabla 1).
- **Google Cloud:** proyecto `mlops-icai-a2265f` con facturación habilitada; clúster `mlops-gke-cluster` en la zona `us-central1-a` con 2 nodos `e2-medium` y GKE 1.36.4.
- **Modelo:** `model.pkl` versionado con DVC en DagsHub; el job de despliegue lo descarga con `dvc pull` antes de construir la imagen de la API.
- El cambio de `train.py` que pide la nota del enunciado de la Unit 5 (`log_model(name=..., skops_trusted_types=[...])`) ya estaba aplicado.

2. Práctica Unit 4: despliegue en GKE

2.1. Configuración de Google Cloud

Se ejecutó con la CLI `gcloud` (en una configuración propia, `mlops`, para no tocar otros proyectos):

```
gcloud projects create mlops-icai-a2265f --name="MLOps ICAI"
gcloud billing projects link mlops-icai-a2265f --billing-account=<cuenta>
gcloud services enable container.googleapis.com artifactregistry.googleapis.com \
    iam.googleapis.com

# Service account para GitHub Actions
gcloud iam service-accounts create modelos --display-name="MLOps CI deployer"
gcloud projects add-iam-policy-binding mlops-icai-a2265f \
    --member=serviceAccount:modelos@mlops-icai-a2265f.iam.gserviceaccount.com \
    --role=roles/container.admin          # Administrador de Kubernetes Engine
gcloud projects add-iam-policy-binding mlops-icai-a2265f \
    --member=serviceAccount:modelos@mlops-icai-a2265f.iam.gserviceaccount.com \
    --role=roles/artifactregistry.admin   # Administrador de Artifact Registry
gcloud iam service-accounts keys create sa-key.json \
    --iam-account=modelos@mlops-icai-a2265f.iam.gserviceaccount.com

gcloud container clusters create mlops-gke-cluster --zone us-central1-a --num-nodes 2
```

Dos pasos adicionales que el enunciado solo menciona en la sección de depuración y que en un proyecto nuevo hacen falta desde el principio:

1. **Repositorio `gcr.io` en Artifact Registry.** Google ha retirado Container Registry; en proyectos nuevos las rutas `gcr.io/<proyecto>/...` solo funcionan si existe un repositorio de Artifact Registry con ese nombre: `gcloud artifacts repositories create gcr.io -repository-format=docker -location=us`.
2. **Permiso de lectura para los nodos.** Los nodos descargan las imágenes con la *compute service account*; se le concedió `roles/artifactregistry.reader` para evitar errores `ImagePullBackOff`.

2.2. Variables y secretos en GitHub

Con `gh` se definieron en *Settings* → *Secrets and variables* → *Actions*:

Nombre	Tipo	Valor
`GCP_PROJECT_ID`	variable	`mlops-icai-a2265f`
`GKE_CLUSTER`	variable	`mlops-gke-cluster`
`GKE_ZONE`	variable	`us-central1-a`
`GCP_SA_KEY`	secret	JSON de la clave de `modelos@...`

La clave JSON nunca se ha guardado en el repositorio; GitHub la enmascara en los logs (se comprobó que ninguno de los logs guardados contiene `private_key`).

2.3. Manifiestos de Kubernetes

Primero se desplegó solo la API (`deployment.yaml` + `service.yaml` de tipo `LoadBalancer`, commit `4902306`). Después, siguiendo la sección 4 del enunciado, se renombraron a `api-*.yaml`, el Service de la API pasó a `ClusterIP` en el puerto 5000 (solo accesible dentro del clúster) y se añadieron el Deployment y el Service públicos de la web (commit `bd9a172`).

`api-deployment.yaml`

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mlops-api-deployment
  labels:
    app: mlops-api
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mlops-api
  template:
    metadata:
      labels:
        app: mlops-api
    spec:
      containers:
      - name: mlops-api
        image: gcr.io/mlops-icai-a2265f/mlops-api:latest
        ports:
        - containerPort: 5000
```

`api-service.yaml`

```yaml
# api-service.yaml -- solo accesible dentro del clúster
apiVersion: v1
kind: Service
metadata:
  name: mlops-api-service
spec:
  selector:
    app: mlops-api
  ports:
    - protocol: TCP
      port: 5000
      targetPort: 5000
  type: ClusterIP
```

`web-deployment.yaml`

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mlops-web-deployment
  labels:
    app: mlops-web
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mlops-web
```

```
  template:
    metadata:
      labels:
        app: mlops-web
    spec:
      containers:
      - name: mlops-web
        image: gcr.io/mlops-icai-a2265f/mlops-web:latest
        ports:
        - containerPort: 8501
        env:
        - name: API_URL
          value: http://mlops-api-service:5000/predict # Conexión al servicio interno de la API
```

web-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: mlops-web-service
spec:
  selector:
    app: mlops-web
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8501
  type: LoadBalancer # El frontend es visible públicamente
```

La web localiza la API por DNS interno de Kubernetes (`http://mlops-api-service:5000/predict`), que `app_web.py` lee de la variable de entorno `API_URL`.

2.4. Job `deploy-to-gke`

Se añadió al workflow `.github/workflows/cml.yaml` un segundo job que depende de `train-and-report`. El listado 1 muestra su versión final (ya con los pasos de Trivy de la Unit 5 y las comprobaciones finales). Respecto al enunciado:

- ***Fix* de CML:** se sustituyó `iterative/setup-cml` por `npm install -g @dvcorg/cml`, como indica el enunciado.
- **`dvc pull`:** se reutiliza la autenticación `basic` con usuario y token de DagsHub que ya usaba el job de entrenamiento.
- **Solo en `push`:** el workflow se dispara con `push` y con `pull_request`; el despliegue se limita a `push` para no desplegar dos veces el mismo commit.
- **`kubectl rollout restart`:** las imágenes usan siempre la etiqueta `latest`; como el spec del Deployment no cambia, `kubectl apply` no reinicia los pods. El reinicio fuerza a descargar la imagen recién subida, y `rollout status` hace que el job falle si los pods no arrancan.
- **Acciones fijadas a commit:** `google-github-actions/auth` y `setup-gcloud` se referencian por SHA (commit `99bff29`) en vez de por etiqueta, igual que las acciones de `iterative` en la práctica anterior. Estas acciones manejan la clave de la service account.

Listing 1: Job `deploy-to-gke` (versión final)

```
  deploy-to-gke:
    needs: train-and-report
    # En un PR el workflow se dispara dos veces (push + pull_request): desplegar solo en push
    if: github.event_name == 'push'
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

```

```yaml
      - name: DVC and Python setup
        uses: iterative/setup-dvc@175771be1dc3d119268e00a896b52a4b77decb5e # v1

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - id: 'auth'
        name: Authenticate to Google Cloud
        uses: google-github-actions/auth@c200f3691d83b41bf9bbd8638997a462592937ed # v2.1.13
        with:
          credentials_json: '${{ secrets.GCP_SA_KEY }}'

      - name: Set up gcloud CLI and kubectl
        uses: google-github-actions/setup-gcloud@e427ad8a34f8676edf47cf7d7925499adf3eb74f # v2.2.1
        with:
          version: 'latest'
          install_components: 'gke-gcloud-auth-plugin,kubectl'

      - name: Get GKE credentials
        env:
          CLOUDSDK_CONTAINER_USE_APPLICATION_DEFAULT_CREDENTIALS: true
        run: gcloud container clusters get-credentials ${{ vars.GKE_CLUSTER }} --zone ${{ vars.GKE_ZONE }} --project ${{ vars.GCP_PROJECT_ID }}

      - name: DVC Pull
        env:
          DAGSHUB_USERNAME: ${{ secrets.DAGSHUB_USERNAME }}
          DAGSHUB_API_KEY: ${{ secrets.DAGSHUB_API_KEY }}
        # Las credenciales de Dagshub son necesarias para acceder al almacenamiento remoto
        run: |
          dvc remote modify origin --local auth basic
          dvc remote modify origin --local user $DAGSHUB_USERNAME
          dvc remote modify origin --local password $DAGSHUB_API_KEY
          dvc pull -r origin

      - name: Configure Docker to use GCR
        run: gcloud auth configure-docker --quiet

      # --- API (mlops-api): construir -> escanear con Trivy -> subir ---
      - name: Build API Docker Image
        id: build_api
        run: |
          IMAGE_NAME="gcr.io/${{ vars.GCP_PROJECT_ID }}/mlops-api:latest"
          docker build -t $IMAGE_NAME .
          echo "IMAGE_API=$IMAGE_NAME" >> $GITHUB_ENV

      # trivy-action fijado a commit SHA (release inmutable v0.36.0) en vez de @master:
      # en marzo de 2026 se secuestraron sus tags (GHSA-69fq-xp46-6x23).
      - name: Trivy Scan for API Image
        uses: aquasecurity/trivy-action@ed142fd0673e97e23eac54620cfb913e5ce36c25 # v0.36.0
        with:
          version: 'v0.76.1'
          image-ref: ${{ env.IMAGE_API }}
          format: 'table'
          exit-code: '1'
          severity: 'CRITICAL'

      - name: Push API Docker Image
        run: docker push ${{ env.IMAGE_API }}

      # --- WEB (mlops-web): construir -> escanear con Trivy -> subir ---
```

```
72      - name: Build Web Docker Image
73        id: build_web
74        run: |
75          IMAGE_NAME="gcr.io/${{ vars.GCP_PROJECT_ID }}/mlops-web:latest"
76          docker build -t $IMAGE_NAME -f Dockerfile.web .
77          echo "IMAGE_WEB=$IMAGE_NAME" >> $GITHUB_ENV
78
79      - name: Trivy Scan for Web Image
80        uses: aquasecurity/trivy-action@ed142fd0673e97e23eac54620cfb913e5ce36c25 # v0.36.0
81        with:
82          version: 'v0.76.1'
83          image-ref: ${{ env.IMAGE_WEB }}
84          format: 'table'
85          exit-code: '1'
86          severity: 'CRITICAL'
87
88      - name: Push Web Docker Image
89        run: docker push ${{ env.IMAGE_WEB }}
90
91      - name: Deploy to GKE
92        run: |
93          # Despliegue de la API
94          kubectl apply -f api-deployment.yaml
95          kubectl apply -f api-service.yaml
96          # Despliegue de la Web
97          kubectl apply -f web-deployment.yaml
98          kubectl apply -f web-service.yaml
99          # :latest no cambia el spec -> forzar que los pods vuelvan a descargar la imagen
100         kubectl rollout restart deployment/mlops-api-deployment deployment/mlops-web-
    deployment
101         kubectl rollout status deployment/mlops-api-deployment --timeout=180s
102         kubectl rollout status deployment/mlops-web-deployment --timeout=180s
103         kubectl get svc
104
105     - name: Smoke test (web -> API inside the cluster, public web IP)
106       run: |
107         # La web llama a la API por el Service interno (ClusterIP), igual que hace Streamlit
108         kubectl exec deploy/mlops-web-deployment -- python -c "import os, requests; r =
    requests.post(os.environ['API_URL'], json={'features': [5.1, 3.5, 1.4, 0.2]}, timeout=10);
    print(r.status_code, r.text); assert r.status_code == 200"
109         # Esperar a que el LoadBalancer de la web tenga IP pública
110         for i in $(seq 1 30); do
111           IP=$(kubectl get svc mlops-web-service -o jsonpath='{.status.loadBalancer.ingress
    [0].ip}')
112           [ -n "$IP" ] && break
113           sleep 10
114         done
115         echo "Web: http://$IP"
116         curl -sf -o /dev/null -w "HTTP %{http_code}\n" "http://$IP/"
117         # Streamlit solo pinta la página si su websocket (/_stcore/stream) conecta: esperar
    101
118         python3 - "$IP" <<'PY'
119         import base64, os, socket, sys
120         ip = sys.argv[1]
121         req = ("GET /_stcore/stream HTTP/1.1\r\nHost: %s\r\nUpgrade: websocket\r\nConnection:
     Upgrade\r\n"
122                "Sec-WebSocket-Key: %s\r\nSec-WebSocket-Version: 13\r\nSec-WebSocket-Protocol:
     streamlit\r\n"
123                "Origin: http://%s\r\n\r\n") % (ip, base64.b64encode(os.urandom(16)).decode()
    , ip)
124         s = socket.create_connection((ip, 80), timeout=10)
125         s.sendall(req.encode())
126         status = s.recv(4096).decode(errors="replace").split("\r\n")[0]
127         print("WebSocket:", status)
```

```
            assert " 101 " in status
            PY
```

2.5. Verificación

API (solo API desplegada, ejecución `41996993196`). El job aplicó los manifiestos y el *rollout* terminó correctamente. La prueba del enunciado contra la IP pública del LoadBalancer de la API devolvió la clase correcta (*Setosa*):

```
$ curl -X POST -H "Content-Type: application/json" \
       -d '{"features": [5.1, 3.5, 1.4, 0.2]}' http://136.64.56.38/predict
{"prediction":0}
```

API + web (ejecución `42004275345`). Salida de `kubectl get svc` al final del job:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	34.118.224.1	<none>	443/TCP	62m
mlops-api-service	ClusterIP	34.118.229.106	<none>	5000/TCP	7m22s
mlops-web-service	LoadBalancer	34.118.227.138	<pending>	80:30107/TCP	18s

La API dejó de ser accesible desde fuera (la antigua IP `136.64.56.38` ya no responde) y la web quedó publicada en `http://35.253.244.201`.

Comprobación web. La IP pública del Service `mlops-web-service` se obtuvo con `gcloud compute forwarding-rules list` (equivale a la columna `EXTERNAL-IP` de `kubectl get svc` o a la consola de GKE). La figura 1 muestra la interfaz servida desde GKE: la web llama a la API por el Service interno y muestra la especie predicha.



Figura 1: Streamlit desplegado en GKE (`http://35.253.244.201`) devolviendo una predicción a través de la API interna.

Prueba automática tras el despliegue. Además se añadió al final del job un paso *Smoke test* (commits `d097049` y `422bbe2`) que falla el pipeline si el despliegue no funciona de extremo a extremo:

1. ejecuta dentro del pod web (`kubectl exec`) una petición a `$API_URL`, es decir, el mismo camino web → API que usa Streamlit;
2. espera a que el LoadBalancer de la web tenga IP y comprueba que responde HTTP 200;
3. hace el *handshake* del websocket de Streamlit (`/_stcore/stream`) y exige `101 Switching Protocols`, ya que sin él la página no llega a pintarse.

Salida del paso en la ejecución final `42007215102`:

```
200 ***"prediction":0***          # web -> API dentro del clúster
Web: http://35.253.244.201
HTTP 200
WebSocket: HTTP/1.1 101 Switching Protocols
```

Las llaves de `{"prediction":0}` aparecen como `***` porque GitHub enmascara cada línea del secreto `GCP_SA_KEY`, y el JSON de la clave tiene líneas que son solo `{` y `}`.

3. Práctica Unit 5: escaneo de imágenes con Trivy

3.1. Puerta de seguridad en el pipeline

En el job `deploy-to-gke` los pasos "construir y subir" de cada imagen se sustituyeron por tres pasos (commit `ce7b369`):

1. **Build**: `docker build` de `gcr.io/<proyecto>/mlops-api:latest` (o `mlops-web`) y se exporta el nombre en `$GITHUB_ENV`.
2. **Trivy**: `aquasecurity/trivy-action` sobre la imagen local con `severity: CRITICAL` y `exit-code: '1'`.
3. **Push**: `docker push`, que solo se ejecuta si el escaneo ha pasado.

Si Trivy encuentra una vulnerabilidad CRITICAL, el paso falla, GitHub Actions salta todos los pasos siguientes (`push` y `Deploy to GKE`) y en el clúster sigue corriendo la última versión buena.

Desviación: `trivy-action` fijada a un commit, no a `@master`

El enunciado usa `aquasecurity/trivy-action@master`. En marzo de 2026 el propio ecosistema de Trivy sufrió un ataque de cadena de suministro (aviso GHSA-69fq-xp46-6x23): los atacantes reescribieron 76 de las 77 etiquetas de `trivy-action` para que ejecutaran un ladrón de credenciales que vuelca la memoria del runner, y publicaron imágenes de Trivy maliciosas (v0.69.4–v0.69.6). En este workflow eso significaría filtrar `GCP_SA_KEY`, con permisos de administrador sobre GKE y Artifact Registry.

Por eso la acción se fija al commit de la release inmutable v0.36.0 (`ed142fd0673e97e23eac54620cfb913e5ce36c25`) y se indica explícitamente la versión de Trivy (`version: 'v0.76.1'`). Para los escaneos locales se usa la imagen `aquasec/trivy:0.76.1` referenciada por *digest*.

3.2. Etapa A: escaneo con las dependencias actuales

El `requirements.txt` del repositorio no fija `mlflow` (se instala la última, 3.18.0), así que no reproduce la tabla del enunciado (`mlflow 2.8.0`, `pyarrow 13.0.0`, ...). En la ejecución `42005391746` Trivy no encontró ninguna vulnerabilidad CRITICAL en ninguna de las dos imágenes, las imágenes se subieron y se desplegaron:

```
gcr.io/mlops-icai-a2265f/mlops-api:latest (debian 13.7)   debian   0
latest: digest: sha256:3433f1089fd5b7616526f9472e2f53f438feafc54b972050ef10a82700b507d9
gcr.io/mlops-icai-a2265f/mlops-web:latest (debian 13.7)   debian   0
latest: digest: sha256:989caf0677b15716a61ac8f6a2424034cda5360fd186ae78473989192b642a09
```

3.3. Etapa B: dependencias vulnerables (la puerta bloquea)

Para comprobar que la puerta bloquea de verdad, se fijaron a propósito las versiones vulnerables de la tabla del enunciado (commit `858489b`):

```
pyarrow==13.0.0
gunicorn==21.2.0
setuptools==65.5.1
```

Por qué no se fijó `mlflow==2.8.0`

`train.py` usa la API de mlflow 3 (`log_model(model, name=..., skops_trusted_types=[...])`), que no existe en 2.8.0, y `requirements.txt` es dependencia de la etapa `train` de DVC: el job `train-and-report` habría fallado antes de llegar al escaneo. Con `pyarrow 13.0.0` ya hay una vulnerabilidad CRITICAL y pip resuelve sin conflictos (`mlflow 3.18.0` acepta esa versión).

En la ejecución `42006388026` el escaneo de la imagen de la API falló y GitHub Actions saltó todos los pasos posteriores; no se subió ninguna imagen y en GKE siguió la versión de la etapa A:

```
Total: 1 (CRITICAL: 1)
| Library | Vulnerability  | Severity | Status | Installed | Fixed  | Title
| pyarrow | CVE-2023-47248 | CRITICAL | fixed  | 13.0.0    | 14.0.1 | PyArrow: Arbitrary
                                                                     code execution when
                                                                     loading a malicious
                                                                     data file
##[error]Process completed with exit code 1.
```

Paso del job `deploy-to-gke`	Resultado
Build API Docker Image	success
Trivy Scan for API Image	failure
Push API Docker Image	skipped
Build / Trivy / Push Web Docker Image	skipped
Deploy to GKE	skipped

CVE-2023-47248 permite ejecutar código arbitrario al leer un fichero IPC/Parquet malicioso, porque `pyarrow` deserializaba con `pickle` el tipo de extensión `arrow.py_extension_type`.

3.4. Etapa C: corrección de `requirements.txt`

Se sustituyeron las versiones vulnerables por las mínimas corregidas de la tabla del enunciado (commit `0468cc0`):

```
pyarrow>=14.0.1      # CVE-2023-47248 (CRITICAL) en 13.0.0
gunicorn>=22.0.0     # vulnerabilidades HIGH en 21.2.0
setuptools>=70.0.0   # vulnerabilidades HIGH en 65.5.1
```

pip instaló `pyarrow 26.0.1`, `gunicorn 26.2.0` y `setuptools 79.0.1` (`mlflow` sigue en 3.18.0). En la ejecución `42007215102` los dos escaneos pasaron, las imágenes se subieron y se desplegaron, y la prueba automática final (sección 2.5) fue correcta:

```
gcr.io/mlops-icai-a2265f/mlops-api:latest (debian 13.7)   debian   0
latest: digest: sha256:af6d16aae23cb83f7d48af5f339117f6dbbb77fed475db00a59f7aa74983aae0
gcr.io/mlops-icai-a2265f/mlops-web:latest (debian 13.7)   debian   0
latest: digest: sha256:864f7c65e4e2dee8b796dc99bfedb58b65c94a8aad78c2eb8fc36a0f55380a0f
deployment "mlops-api-deployment" successfully rolled out
deployment "mlops-web-deployment" successfully rolled out
```

3.5. Escaneo local (critical y high)

El enunciado propone ejecutar Trivy en local con `-severity CRITICAL,HIGH`. En Windows/WSL se usó el contenedor oficial; al comando del enunciado le falta montar el socket de Docker, sin el cual Trivy no ve las imágenes locales:

```
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
  -v $HOME/trivy-cache:/root/.cache/ \
  aquasec/trivy:0.76.1@sha256:790fd66d1697395d8693d1acaa1ea702406a20f109b6fc7bac8db623eca6c06c \
  image --severity CRITICAL,HIGH --exit-code 1 mlops-web:latest
```

Imagen	Dependencias	CRITICAL	HIGH (Debian)	HIGH (Python)
`mlops-api`	actuales	0	48	2
`mlops-web`	actuales	0	48	2
`mlops-web`	vulnerables (etapa B)	1	—	—
`mlops-web`	corregidas (etapa C)	0	48	2

Las 48 vulnerabilidades HIGH restantes son paquetes del sistema de la imagen base `python:3.11-slim` (Debian 13.7) y las 2 de Python están en librerías que `setuptools` lleva dentro (`jaraco.context`, `wheel`). No dependen del `requirements.txt`; para eliminarlas habría que actualizar la imagen base o instalar parches con `apt-get upgrade` en el Dockerfile. Con el umbral del pipeline (CRITICAL) no bloquean el despliegue.

4. Problemas encontrados

GitHub Actions bloqueado por facturación

Los primeros intentos fallaban antes de empezar con *"The job was not started because your account is locked due to a billing issue"*. La cuenta tenía un presupuesto de Actions de 0 $ con "stop usage", y un consumo mínimo ($< 0{,}01$ $, cubierto por el uso incluido) bastaba para marcarlo al 100 %. Se resolvió desde la configuración de facturación de GitHub.

La web se queda cargando desde la red de la universidad

La página HTML y el JavaScript de Streamlit cargaban (HTTP 200), pero la interfaz no se pintaba. Streamlit necesita abrir un websocket en `/_stcore/stream`; desde la red local esa conexión se cortaba siempre (*connection reset*). La misma imagen en local respondía `101 Switching Protocols`, y servidores de eco websocket públicos en el puerto 80 también eran cortados desde esa red, así que la causa es un filtro de la red (websockets sin TLS en el puerto 80), no el despliegue. Para comprobarlo desde fuera se añadió al pipeline una prueba que hace el *handshake* desde el runner de GitHub (sección 2.5).

`kubectl` local sin `gke-gcloud-auth-plugin`

No fue necesario: todo el despliegue lo hace el pipeline y las IPs públicas se consultaron con `gcloud compute forwarding-rules list`.

5. Resumen de ejecuciones

6. Costes y limpieza

El clúster (2 nodos `e2-medium`) y los dos balanceadores cuestan del orden de 3–5 $ al día. Tras la corrección se elimina el clúster con `gcloud container clusters delete mlops-gke-cluster -zone us-central1-a`; el proyecto, la service account y las imágenes en `gcr.io` se conservan.

Ejecución	Commit	Contenido	Resultado
`41996993196`	`99bff29`	Solo API en GKE (LoadBalancer)	OK
`42004275345`	`bd9a172`	API (ClusterIP) + web (LoadBalancer)	OK
`42005391746`	`ce7b369`	Puerta Trivy, dependencias actuales	OK
`42006388026`	`858489b`	Dependencias vulnerables (`pyarrow 13.0.0`)	Bloqueado
`42007215102`	`422bbe2`	Dependencias corregidas + prueba automática	OK

Cuadro 1: Ejecuciones de GitHub Actions en la rama `p6-k8s-trivy`.

7. Conclusiones

- Cada *push* a la rama entrena el modelo, construye las dos imágenes, las escanea y, solo si están limpias, las sube a `gcr.io` y las despliega en GKE.
- La API queda interna (`ClusterIP`) y solo la web es pública, lo que reduce la superficie expuesta.
- La puerta de Trivy cumple su función: una dependencia con una vulnerabilidad crítica de ejecución de código (`pyarrow 13.0.0`) no llega al registro ni al clúster.
- La propia herramienta de seguridad es parte de la cadena de suministro: fijar acciones y herramientas a commits/digests inmutables es tan importante como escanear. Como mejora, la clave JSON de larga duración podría sustituirse por *Workload Identity Federation*, de modo que no haya ningún secreto que robar.